Recommendation System Reimagined

From Loss Paradigms to Temporal Dynamics and Beyond

Arnab Roy

22-03-2026

Outline

1. Introduction
2. Loss Paradigm
3. Temporal Dynamics
4. Sequential Recommendation
5. Future Work

Problem Statement & Objectives

- **The Core Problem:** Modern recommender systems are trained on static snapshots of user behaviour — but user preferences, item popularity, and interaction patterns evolve continuously over time.
  - A model trained at time T becomes increasingly stale as it is deployed beyond T
  - Standard Matrix Factorisation treats all historical interactions equally — ignoring **recency** and **temporal drift**
- **What We Investigate:**
  - Does the choice of **loss paradigm** fundamentally change what the model learns?
  - Can we empirically **quantify preference drift** and link it directly to degrading recommendation quality?
  - How do **temporal-aware models** address these limitations?
  - What role do **Graph Neural Networks** play in capturing structural user-item relationships?
- **Dataset:** MovieLens 32M — $\sim$32M ratings, $\sim$200K users, $\sim$87K movies spanning 1995–2023

Why Latent Factor Models?

- **Our Data:** Explicit pairwise ratings — each observation is a triple (u, i, r_{ui}) where user u rated item i with score $r_{ui} \in \{0.5, 1.0, \ldots, 5.0\}$
  - $\sim$32M such triples, but the full user$\times$item matrix is **99.8% sparse** — most users have never rated most movies
- **Why Latent Factor Models?** We need to generalise across this sparsity — the idea is to explain ratings through a small number of latent factors (genre taste, recency preference, production quality, etc.) that are **never observed directly**
- **How it works:** Learn two embedding matrices —

$$P \in \mathbb{R}^{|U| \times d} \quad \text{(users)}, \qquad Q \in \mathbb{R}^{|I| \times d} \quad \text{(items)}$$

  Predicted rating is:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathrm{p}_u^\top \mathrm{q}_i$$

  where μ is the global mean, b_u and b_i are user and item bias terms
- **Training:** Minimise a loss $\mathcal{L}(r_{ui}, \hat{r}_{ui})$ over all observed ratings via SGD — the original Netflix Prize paper [1] recommends **MSE** as the natural choice for explicit rating data

Koren, Bell, Volinsky. *Matrix Factorization Techniques for Recommender Systems.* IEEE Computer, 2009. [PDF]

Two Losses, One Algorithm

- We train the same Matrix Factorisation algorithm on **MovieLens 32M** using a **6-month interaction window**
  - Same architecture, same hyperparameters, same data — only the loss function changes
- We experiment with two fundamentally different loss functions — **Loss A** and **Loss B**
  - They optimise for different things, and we let the results speak for themselves
- We ask a simple but deep question:

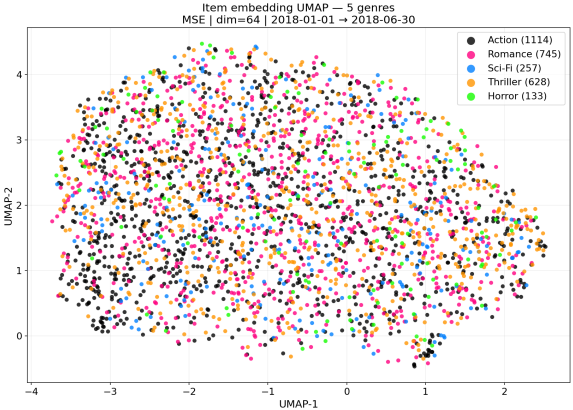
"If you change what the model is punished for, do the embeddings it learns tell a different story about movies and users?"

- Over the next few slides we will examine the learned embeddings through three lenses — **geometry**, **purity**, and **user taste profiles** — before finally revealing which loss wins on recommendation quality

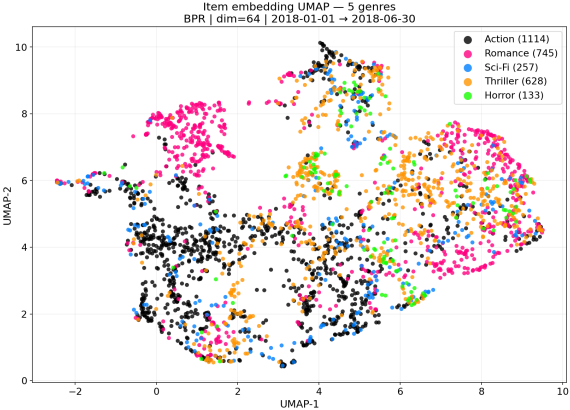
Do the Embeddings Know What Genre a Movie Is?

Dimensionality Reduction of Movie Embeddings (UMAP, Top 5 Genres)

Loss A



Loss B



Genre Purity — Quantifying the Geometry

Can embeddings organise movies by genre — without ever being told what genre is?

- For each movie i, find its K nearest neighbours in embedding space
- Count how many neighbours share at least one genre with movie i
- Average this fraction across all movies:

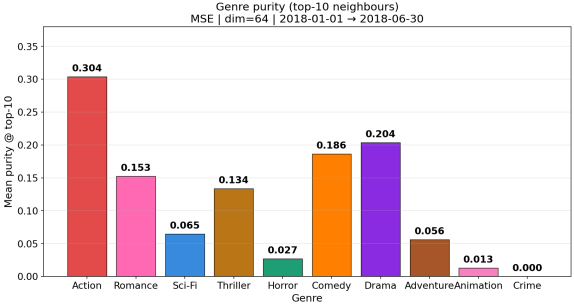
$$\text{Purity@K} = \frac{1}{|I|} \sum_{i \in I} \frac{|\{j \in \mathcal{N}_K(i) : \mathcal{G}_i \cap \mathcal{G}_j \neq \emptyset\}|}{K}$$

- $\mathcal{N}_K(i)$ — K nearest neighbours of movie i in $\mathbb{R}^{64}$
- $\mathcal{G}_i$ — genre set of movie i
- Score $\in [0, 1]$ — higher means tighter genre clustering

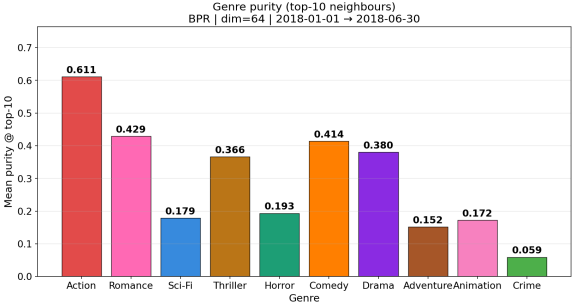
A purely unsupervised signal — genre structure was never part of training.

Genre Purity — Loss A vs Loss B

Loss A



Loss B



- Both models learned **genre-aware geometry** without supervision
- The gap between Loss A and Loss B tells us which loss encourages tighter semantic clustering
- We are still not revealing which is which — the case studies on the next slides will build the full picture

User Taste Profiles — Can Embeddings Capture Who You Are?

What is a Genre Fingerprint Radar?

- For a **query user**, compute their normalised genre distribution from rating history — this is their *taste fingerprint*
- Find their **2 most similar** and **2 most dissimilar** users *purely from embedding distance* in $\mathbb{R}^{64}$
- Plot all five fingerprints on a radar chart — each axis is a genre, each value is the fraction of that genre in the user's history

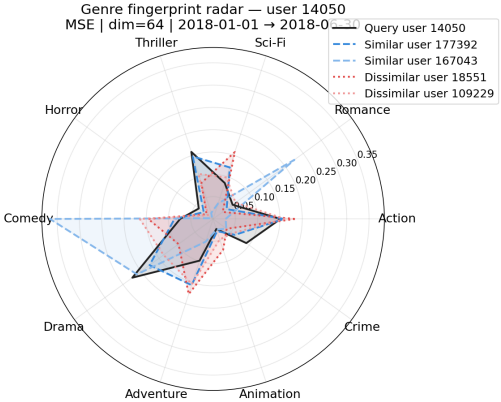
$$\text{fingerprint}_u[g] = \frac{\text{ratings by } u \text{ on genre } g}{\text{total ratings by } u}$$

- Similarity and dissimilarity are computed from **cosine distance** in embedding space — genre labels play no role

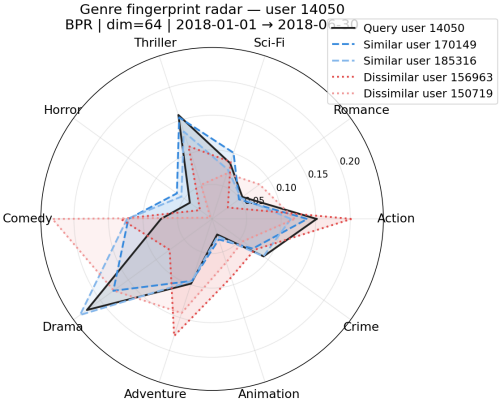
If users close in embedding space share similar taste profiles — the model has learned something real about human preferences.

User 14050 — Loss A vs Loss B

Loss A



Loss B



Black — query user · **Blue dashed** — similar users · **Red dotted** — dissimilar users

Do the blue profiles mirror the black? Do the red profiles diverge from it?

How Does the System Recommend?

- After training, for a given user u we generate a ranked list of unseen movies
- The score for any unseen movie i is simply the predicted rating:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

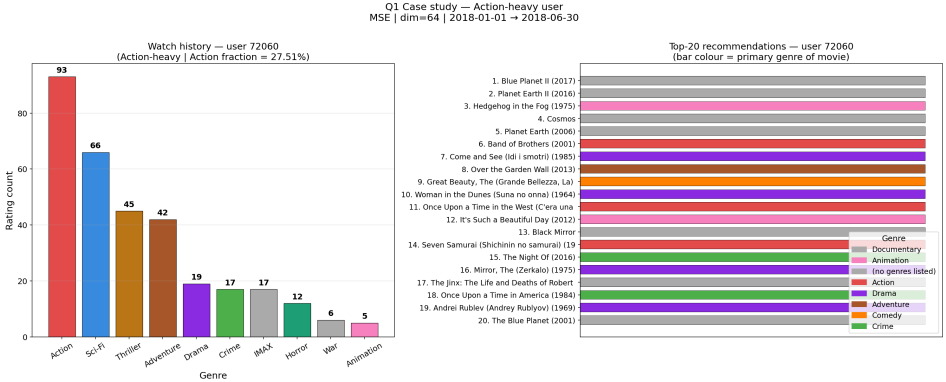
- Movies already seen during training are **excluded** from the candidate set
- The top N highest scoring unseen movies become the recommendation list

The Case Study Setup

- We identify **genre-heavy users** — users whose rating history is dominated by a single genre (Action, Romance, Horror, Sci-Fi, Thriller)
- We ask: does the recommendation list **reflect the user's taste profile**?
- Loss A and Loss B are compared side by side — same user, same candidate pool, different embeddings

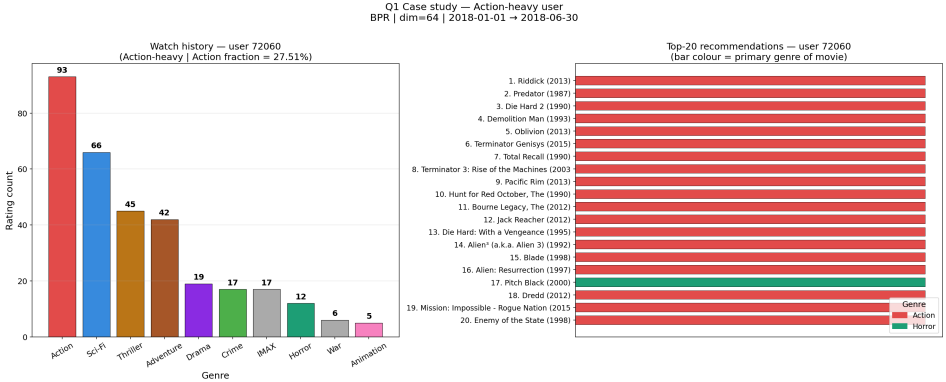
A good recommender should know what you love — without being told explicitly.

Case Study — Action Heavy Users | Loss A



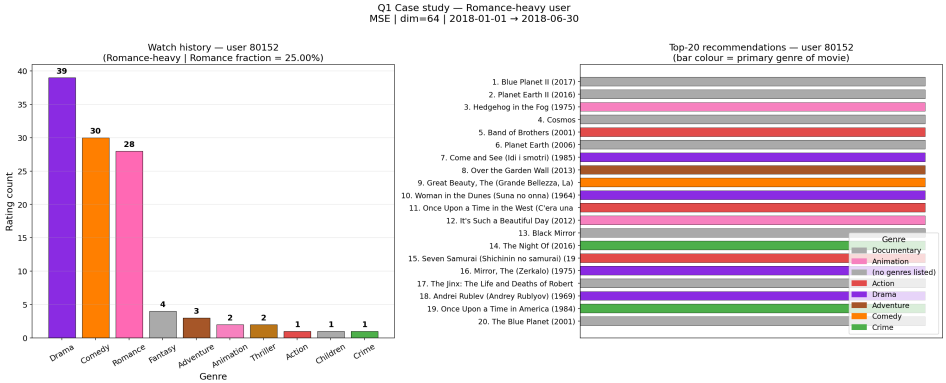
For users whose history is **dominated by Action movies** — does Loss A's recommendation list mirror that preference?

Case Study — Action Heavy Users | Loss B



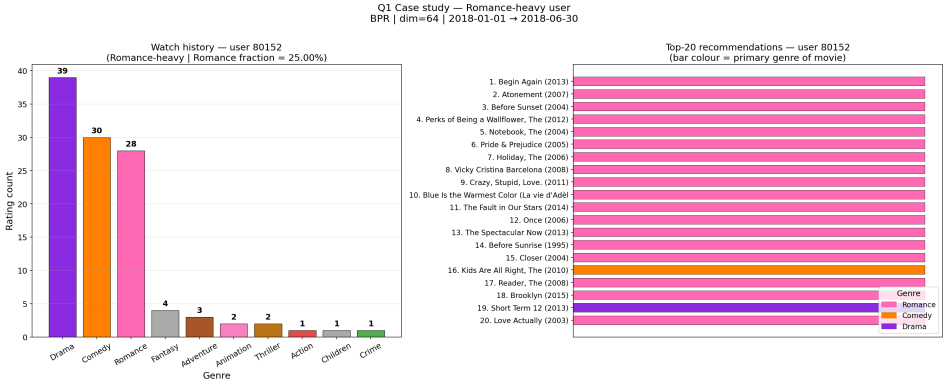
Same users, same candidate pool — now through the lens of Loss B. *Does the recommendation list tell a different story?*

Case Study — Romance Heavy Users | Loss A



Romance is a niche taste — a harder test for any embedding. How does Loss A handle a user with a very specific preference?

Case Study — Romance Heavy Users | Loss B



Same niche users — Loss B's turn. *Which loss stays true to what the user actually watches?*

Revealing the Losses — A and B Unveiled

Loss A — Mean Squared Error (MSE)

$$\mathcal{L}_{\text{MSE}} = \sum_{(u,i)\in\mathcal{R}} (r_{ui} - \hat{r}_{ui})^2 + \lambda\|\Theta\|^2$$

- Treats recommendation as a **rating prediction** problem
- Penalises every deviation from the true rating equally
- Optimises for **absolute score accuracy** — not whether the right movie ends up at the top

Loss B — Bayesian Personalised Ranking (BPR) [1]

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)\in\mathcal{D}} \log\sigma(\hat{r}_{ui} - \hat{r}_{uj}) + \lambda\|\Theta\|^2$$

- i — item user **interacted with** (positive)
- j — item user **never saw** (negative sample)
- Optimises for **relative order** — not absolute scores

Why BPR Makes Intuitive Sense

- BPR asks one simple question per training step:

> *"Should movie i rank higher than movie j for user u?"*

- This is **contrastive learning** — the model learns by comparing a positive against a negative, not by predicting an absolute value
- The same idea powers **Word2Vec** — a word is defined by what surrounds it vs. what does not
- In both cases the signal comes from **contrast**, not from a regression target
- MSE never sees an unrated item — BPR actively **learns what to avoid**

The Contrastive Intuition

Word2Vec: *"bank"* appears near *"river"* but not near *"surgery"*

BPR: user u watched movie i but not movie j

$$\Downarrow$$

Push positive closer, pull negative away

The loss $-\log \sigma(\hat{r}_{ui} - \hat{r}_{uj})$ is maximised when $\hat{r}_{ui} \gg \hat{r}_{uj}$ — the model is rewarded purely for **getting the order right**

The Open Problem — Negative Sampling

- BPR requires a **negative item** j for every positive interaction (u, i)
- We sample **4 negatives per positive** — unrated movies drawn proportional to global popularity, so negatives are unseen but well-known.

Why Negative Sampling Matters

A trivially easy negative (an obscure movie no one has heard of) teaches the model nothing. A **hard negative** (a popular movie in a genre the user dislikes) forces the model to learn finer distinctions — and drives significantly better embeddings.

- Negative sampling strategy in RecSys is an **active area of research**
- Recent work on **dynamic negative sampling** [2] shows that adaptively selecting harder negatives during training leads to substantial gains over static random sampling

Rendle et al. *BPR: Bayesian Personalised Ranking from Implicit Feedback*. UAI 2009. [PDF]

Dynamic Negative Sampling for Recommender Systems. 2023. [PDF]

How Do We Evaluate a Recommender System Offline?

- We cannot deploy the model and wait for real user feedback — instead we **simulate the future**
- **Temporal holdout strategy:** train on interactions up to time T, evaluate on interactions in $(T, T+1], (T+1, T+2], \ldots$

Recall@K

$$\text{Recall@K} = \frac{1}{|U|} \sum_u \frac{|\text{top-}K_u \cap \mathcal{G}_u|}{\min(|\mathcal{G}_u|, K)}$$

How many of the movies the user **actually watched** in the eval window appear in the top-K recommendations?

NDCG@K

$$\text{NDCG@K} = \frac{1}{|U|} \sum_u \frac{\text{DCG@K}_u}{\text{IDCG@K}_u}$$

Same as Recall@K but **rank-aware** — a correct recommendation at rank 1 is worth more than one at rank K

$\mathcal{G}_u$ — ground truth movies watched by user u in the eval window · Embeddings are **frozen** at training time T

NDCG@K — A Worked Example

Setup

- User watched 3 movies in the eval window — A, B, C
- Model returns a top-5 ranked list
- ✓ = hit × = miss

Rank	Movie	Hit?	Gain
1	A	✓	$\frac{1}{\log_2 2} = 1.000$
2	Movie X	×	0
3	B	✓	$\frac{1}{\log_2 4} = 0.500$
4	Movie Y	×	0
5	C	✓	$\frac{1}{\log_2 6} = 0.387$

Computing NDCG@5

$$\text{DCG@5} = 1.000 + 0 + 0.500 + 0 + 0.387 = 1.887$$

Ideal case — all 3 hits at the very top:

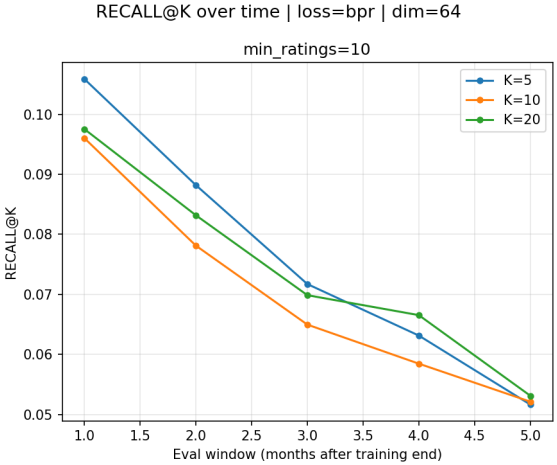
$$\text{IDCG@5} = \frac{1}{\log_2 2} + \frac{1}{\log_2 3} + \frac{1}{\log_2 4} = 2.131$$

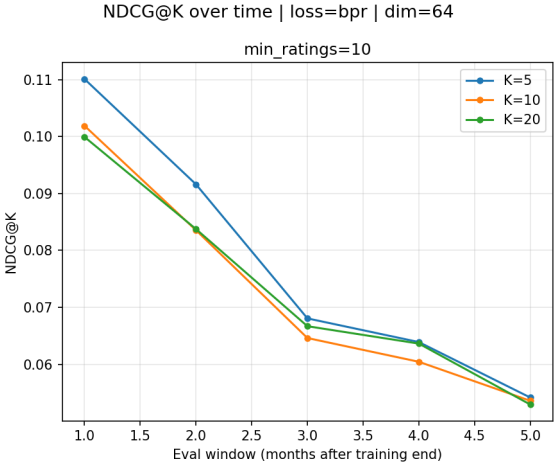
$$\boxed{\text{NDCG@5} = \frac{1.887}{2.131} = 0.885}$$

The key insight

Recall@5 would give $\frac{3}{3} = 1.0$ — a perfect score despite B and C being buried at ranks 3 and 5. NDCG **penalises late hits**.

The Temporal Story — Metrics Degrade Over Time





- Model trained on **6 months** of interactions — embeddings frozen at month T, Evaluated month by month: $T+1, T+2, \ldots, T+5$

Measuring Temporal Drift — Two Metrics

Metric 1 — Item Popularity Drift

- Each month t, collect the top-N most rated movies $\mathcal{P}_t$
- Measure how far the popular item set has rotated away from the baseline month t_0:

$$\delta_t^{\text{item}} = 1 - \frac{|\mathcal{P}_t \cap \mathcal{P}_{t_0}|}{|\mathcal{P}_t \cup \mathcal{P}_{t_0}|}$$

- $\delta_{t_0}^{\text{item}} = 0$ by definition
- Rising score $\Rightarrow$ popular movies are rotating away from what the model was trained on

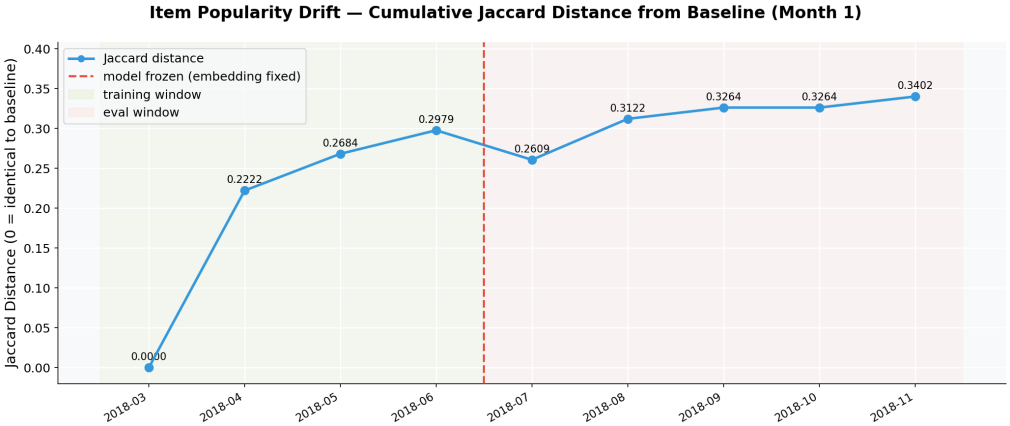
Metric 2 — User Preference Drift

- Each month t, compute per-user genre distribution g_u^t for users active in both t_0 and t
- Measure average divergence from baseline taste:

$$\delta_t^{\text{user}} = \frac{1}{|\mathcal{U}_{t_0 \cap t}|} \sum_{u \in \mathcal{U}_{t_0 \cap t}} \mathrm{JSD}\left(\mathrm{g}_u^{t_0} \,\|\, \mathrm{g}_u^t\right)$$

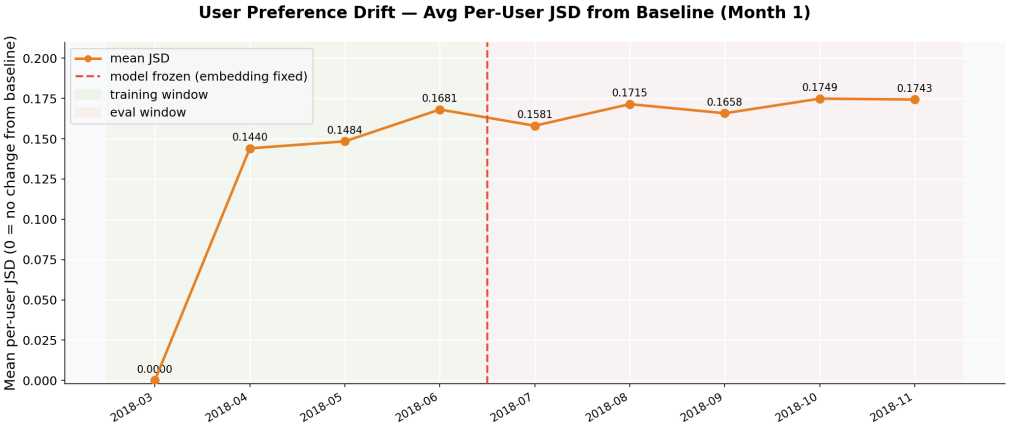
- $\mathcal{U}_{t_0 \cap t}$ — users active in both t_0 and t
- $\delta_{t_0}^{\text{user}} = 0$ by definition
- Rising score $\Rightarrow$ individual users are watching different genres than when the model was trained

The World Kept Moving — The Model Did Not



Rising curve = the set of popular movies is rotating away from what the model was trained on.

The World Kept Moving — The Model Did Not



Rising curve = individual users are watching different genres compared to when the model was trained.

The Fundamental Tradeoff

What the degradation tells us

Recency matters — user preferences and item popularity shift continuously over time

If we tighten the training window

- + Captures recent preferences more faithfully
- + Less stale at deployment
- – Far fewer interactions to learn from
- – Sparse users and items drop out entirely
- – Embeddings become noisy and unstable

If we widen the training window

- + Rich signal, stable embeddings
- + Better coverage of long-tail users and items
- – Old interactions dilute recent preferences
- – Model learns a **blurred average** of who the user was — not who they are today

Breaking the Tradeoff — What If We Never Compress?

The root cause of the tradeoff

MF compresses a user's entire history into a single frozen vector p_u — all temporal signal is lost at that moment

MF approach

- × Compress history $\rightarrow p_u \in \mathbb{R}^{64}$ at train time
- × p_u is frozen at deployment
- × Wider window = more drift baked in
- × Tighter window = less signal
- × No way out of the tradeoff

Sequential approach

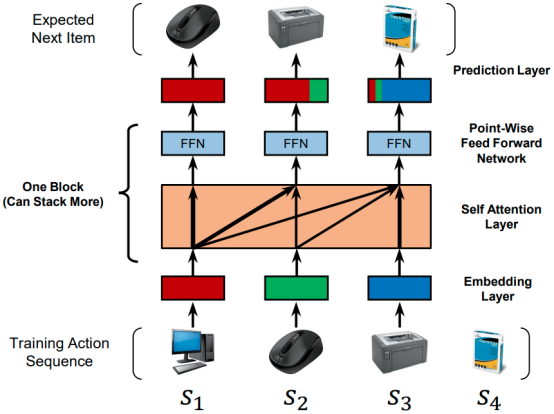
- ✓ Store the full interaction sequence
- ✓ **Re-read the sequence at inference time**
- ✓ Attention naturally focuses on recent, relevant items
- ✓ No frozen user vector — the user representation is always fresh
- ✓ Longer history = more pattern, not more blur

SASRec — Self-Attentive Sequential Recommendation [1]

- Given a user's interaction history as an ordered sequence:

$$[m_1, m_2, \ldots, m_L] \xrightarrow{\text{predict}} m_{L+1}$$

- Each item is embedded and fed through a **Transformer encoder** with **causal masking** — position t can only attend to positions $\leq t$
- The final hidden state h_L captures the user's **evolving intent** up to the last interaction
- **Our configuration:** 2 layers, 2 attention heads, $d = 64$, max sequence length $= 50$



Kang & McAuley. *Self-Attentive Sequential Recommendation.* ICDM 2018.

Result 1 — Short Window: Is the Complexity Worth It?

Training window: 2018-01-01 → 2018-06-30

Model	K	Recall	NDCG
MF (BPR)	5	0.1059	**0.1101**
SASRec	5	0.1191	**0.1207**
MF (BPR)	10	0.0960	**0.1019**
SASRec	10	0.1082	**0.1126**
MF (BPR)	20	0.0975	**0.0999**
SASRec	20	**0.1142**	0.1136

- SASRec shows a **slight but consistent gain** over MF on a 6-month window
- The improvement is modest — one might question whether the added complexity of a Transformer is justified
- **But there is a twist:** Transformers are **data-hungry models** — a 6-month window provides limited sequence length per user
- The sequential signal is present but the model has not seen enough history to fully exploit it

What happens when we give SASRec a longer history to read?

Result 2 — Longer Window: A Gamechanger

Training windows: 2016-06-30 → 2018-06-30 and 2017-01-01 → 2018-12-31

2016-06-30 → 2018-06-30

Model	NDCG@5	NDCG@10
MF (BPR)	0.0859	0.0812
SASRec	**0.1569**	**0.1393**
Gain		**+82%**

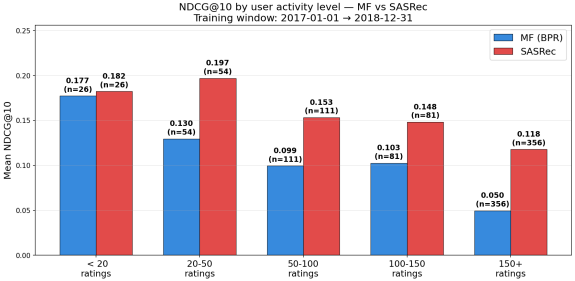
2017-01-01 → 2018-12-31

Model	NDCG@5	NDCG@10
MF (BPR)	0.0821	0.0774
SASRec	**0.1471**	**0.1326**
Gain		**+79%**

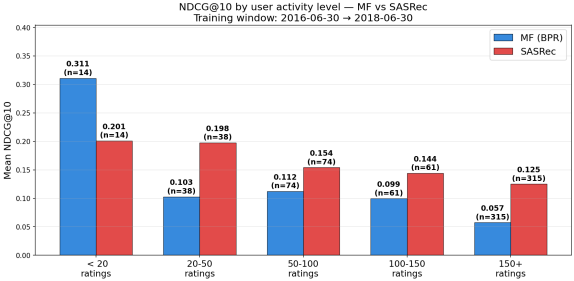
MF performance degrades on longer windows — more history blurs the signal. **SASRec improves dramatically** — more history gives it more pattern to read.

Why? — Sequence Length is the Key

2017-01-01 → 2018-12-31

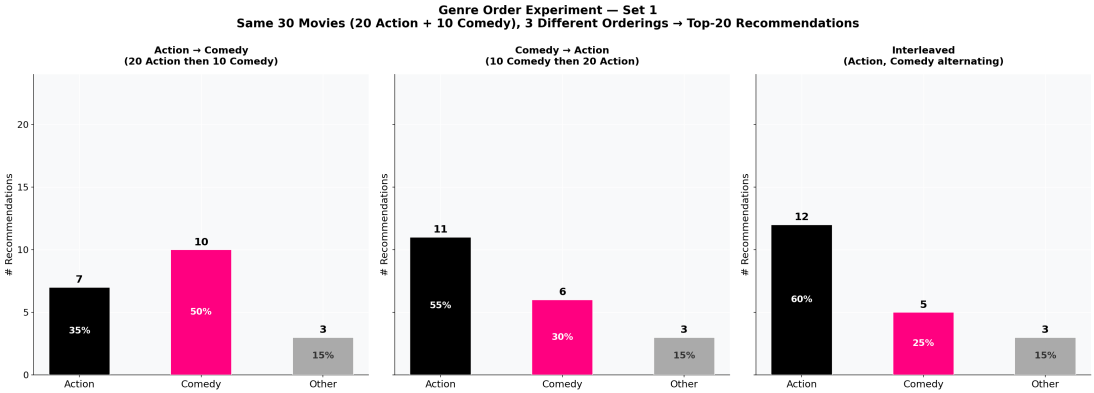


2016-06-30 → 2018-06-30



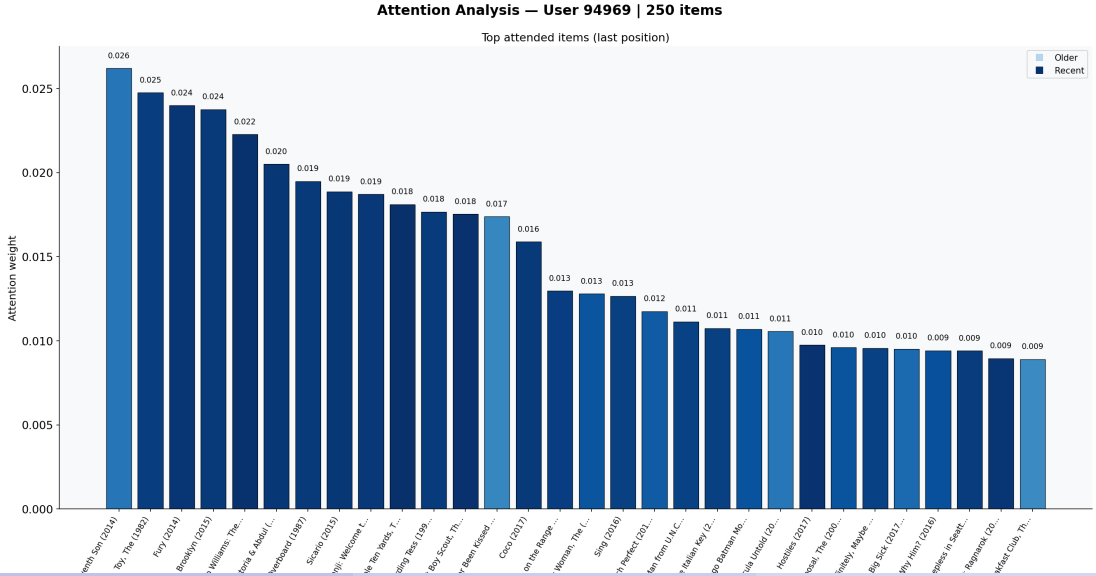
- **Short sequences (<20 ratings):** MF is competitive or better — not enough history for SASRec to detect a pattern
- **Long sequences (150+ ratings):** MF collapses to ∼0.05 NDCG — the blurred average effect. SASRec stays strong at ∼0.12 — it reads the recent trend, not the global average
- **The gap widens with sequence length**. MF is hurt by long history; SASRec is powered by it

Order Matters — Better Personalisation

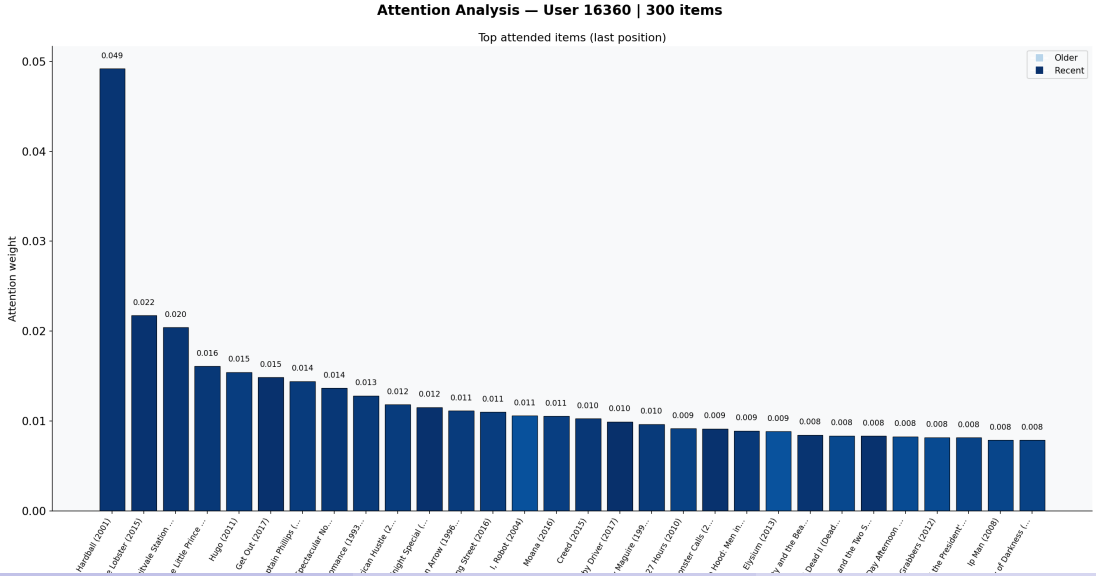


- Same 30 movies (20 Action + 10 Comedy) fed in 3 different orderings — top-20 recommendations change significantly
- **Action → Comedy:** model detects recent Comedy interest → 50% Comedy recs
- **Comedy → Action:** model detects recent Action interest → 55% Action recs

What Does SASRec Actually Attend To? — User 94969



What Does SASRec Actually Attend To? — User 16360



Thank You